

GPUMODE Lecture Study Notes: Lecture 20

Scan Algorithm

Core Problem the Lecture Addresses

The lecture studies the **parallel prefix sum**, also called **scan**, on GPUs. Scan is a fundamental parallel primitive because it converts local counts, flags, or partial summaries into global offsets or prefix values. The specific engineering problem is:

How do we compute all prefix sums of an array in parallel, while respecting data dependences that make the sequential version inherently loop-carried?

For an input array

$$X = [x_0, x_1, \dots, x_{n-1}],$$

and an associative binary operator $\oplus$, scan produces an output array

$$Y = [y_0, y_1, \dots, y_{n-1}].$$

There are two common variants:

$$\text{inclusive scan:} \quad y_i = x_0 \oplus x_1 \oplus \dots \oplus x_i, \quad (1)$$

$$\text{exclusive scan:} \quad y_i = e \oplus x_0 \oplus x_1 \oplus \dots \oplus x_{i-1}, \quad (2)$$

where e is the identity element of the operator. For addition, $e = 0$.

The lecture focuses on implementing scan inside a single CUDA thread block using shared memory, synchronization, and increasingly better parallel algorithms. The main algorithms discussed are:

- the sequential scan,
- the segmented multi-block GPU scan strategy,
- the Kogge–Stone style scan,
- double buffering to remove a false dependence,

- the work-efficiency problem of Kogge–Stone,
- the Brent–Kung style scan as a more work-efficient alternative.

Required Background

Associative Operators

Scan requires the operator $\oplus$ to be **associative**:

$$(a \oplus b) \oplus c = a \oplus (b \oplus c).$$

Associativity matters because a parallel implementation changes the order in which partial results are grouped. Common scan operators include:

- addition: $a \oplus b = a + b$,
- multiplication: $a \oplus b = ab$,
- minimum: $a \oplus b = \min(a, b)$,
- maximum: $a \oplus b = \max(a, b)$.

For floating-point addition, mathematical associativity is only approximate because of rounding. GPU scan algorithms still use floating-point addition in practice, but the result may differ slightly from a strict sequential order.

Inclusive Versus Exclusive Scan

For the input

$$X = [3, 6, 7, 4, 8, 2, 1, 9],$$

the inclusive addition scan is

$$Y_{\text{inclusive}} = [3, 9, 16, 20, 28, 30, 31, 40].$$

The exclusive addition scan is

$$Y_{\text{exclusive}} = [0, 3, 9, 16, 20, 28, 30, 31].$$

The exclusive scan is especially useful for computing write offsets. Suppose thread i owns x_i output items. Then an exclusive scan over the item counts tells each thread where its output segment begins.

For example, if the per-thread item counts are

$$[3, 6, 7, 4],$$

then the exclusive scan is

$[0, 3, 9, 16]$.

So thread 0 writes starting at index 0, thread 1 writes starting at index 3, thread 2 writes starting at index 9, and thread 3 writes starting at index 16.

CUDA Execution Model

The lecture relies on the following CUDA hierarchy:

- A **grid** contains many thread blocks.
- A **thread block** contains many threads that can synchronize with `__syncthreads()`.
- Threads in the same block can communicate through **shared memory**.
- Threads in different blocks cannot cheaply synchronize inside a single ordinary CUDA kernel.

This matters because scan has dependences. It is much cheaper to coordinate threads inside a block than across the whole grid.

Memory Hierarchy

The implementation uses:

- **global memory** for the input array, output array, and inter-block partial sums,
- **shared memory** for the per-block scan buffer,
- **registers** for thread-local temporary values.

The key optimization is to load a block-sized segment from global memory into shared memory, perform the iterative scan in shared memory, then write the final values back to global memory.

Main Concepts

Sequential Scan

Sequential inclusive scan is simple:

```
output[0] = input[0];

for (unsigned int i = 1; i < n; ++i) {
    output[i] = output[i - 1] + input[i];
}
```

The recurrence is

$$y_i = y_{i-1} + x_i.$$

The problem is the **loop-carried dependence**. Iteration i cannot compute y_i until iteration $i - 1$ has computed y_{i-1} . This makes the naive sequential loop unsuitable for direct one-thread-per-element GPU parallelization.

For exclusive scan, one common sequential form is:

```
output[0] = 0;

for (unsigned int i = 1; i < n; ++i) {
    output[i] = output[i - 1] + input[i - 1];
}
```

The recurrence is

$$y_i = y_{i-1} + x_{i-1}.$$

Segmented Multi-Block Scan Strategy

A full GPU array may contain millions of elements, so one CUDA block cannot scan the whole array. The lecture introduces a segmented strategy:

1. Split the input array into block-sized segments.
2. Each thread block scans one segment independently.
3. Each block writes its segment sum into a separate partial-sums array.
4. Scan the partial-sums array.
5. Add the scanned partial sum of previous blocks to every element in the corresponding segment.

If block b owns segment S_b , it first computes a local scan:

$$Y_{\text{local}}^{(b)}.$$

Let the sum of segment b be

$$p_b = \sum_{x \in S_b} x.$$

Then the correction that block b must add is the exclusive scan of the previous segment sums:

$$c_b = \sum_{j=0}^{b-1} p_j.$$

The final output for an element inside block b is

$$Y_{\text{final}}^{(b)} = Y_{\text{local}}^{(b)} + c_b.$$

This reveals a recursive structure: scanning a large array requires scanning the partial sums of blocks.

Why the Lecture Focuses on One Thread Block

The lecture deliberately focuses on the intra-block scan. Cross-block coordination is handled using multiple kernel launches:

1. kernel 1: scan each block segment and produce block partial sums,
2. kernel 2: scan the partial sums,
3. kernel 3: add scanned partial sums back into each block segment.

A kernel launch boundary acts as a global synchronization point. This is expensive compared with block-local synchronization, but it is straightforward and correct.

Kogge–Stone Style Parallel Scan

Reduction Tree Intuition

One way to compute a prefix sum is to imagine many reduction trees overlaid on top of each other.

For an eight-element input

$$[x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7],$$

the final inclusive scan should be

$$[x_0, x_0 + x_1, x_0 + x_1 + x_2, \dots, x_0 + x_1 + \dots + x_7].$$

A normal reduction tree can compute the final total

$$x_0 + x_1 + \dots + x_7,$$

but scan needs every prefix, not only the final total. Kogge–Stone style scan overlays several partial reduction trees so that each element accumulates values from increasing distances behind it.

Stride-Doubling Pattern

At each step, the stride doubles:

$$\text{stride} \in \{1, 2, 4, 8, \dots\}.$$

At stride s , thread i updates its value using the value at index $i - s$:

$$y_i^{(k+1)} = \begin{cases} y_i^{(k)} + y_{i-s}^{(k)}, & i \geq s, \\ y_i^{(k)}, & i < s. \end{cases}$$

Here

$$s = 2^k.$$

For n elements, the number of steps is

$$\log_2 n.$$

After the final stride, each element has accumulated all preceding values needed for its prefix.

Example for Eight Elements

Initial state:

$$[x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7].$$

After stride 1:

$$[x_0, x_0 + x_1, x_1 + x_2, x_2 + x_3, x_3 + x_4, x_4 + x_5, x_5 + x_6, x_6 + x_7].$$

After stride 2:

$$[x_0, x_0+x_1, x_0+x_1+x_2, x_0+x_1+x_2+x_3, x_1+x_2+x_3+x_4, x_2+x_3+x_4+x_5, x_3+x_4+x_5+x_6, x_4+x_5+x_6+x_7].$$

After stride 4:

$$[x_0, x_0+x_1, x_0+x_1+x_2, x_0+x_1+x_2+x_3, x_0+\dots+x_4, x_0+\dots+x_5, x_0+\dots+x_6, x_0+\dots+x_7].$$

Visual Concept

Draw eight boxes in a row for the input elements. Then draw three layers of arrows: stride 1 arrows from each element to its right neighbor, stride 2 arrows skipping one element, and stride 4 arrows skipping three elements. Label the layers as synchronization steps.

Hardware-Level Explanation

Mapping Threads to Elements

The basic one-block implementation maps one CUDA thread to one array element in the block segment.

The global index is

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

The local shared-memory index is

$$t = \text{threadIdx.x}.$$

Thread t loads

$$\text{input}[i]$$

into

$$\text{buffer_s}[t].$$

This gives coalesced global memory loads when adjacent threads access adjacent input elements.

Shared Memory Buffer

The lecture uses shared memory because repeated global-memory reads and writes would be slow. A block loads its segment once:

$$\text{global memory} \rightarrow \text{shared memory},$$

performs all stride-doubling updates in shared memory, then writes once:

$$\text{shared memory} \rightarrow \text{global memory}.$$

The shared-memory buffer is block-sized:

```
buffer_s[BLOCK_DIM].
```

For a block size of 1024, a single float buffer consumes

$$1024 \cdot 4 = 4096 \text{ bytes.}$$

Double buffering uses two such buffers:

$$2 \cdot 4096 = 8192 \text{ bytes.}$$

This is usually acceptable on modern GPUs, but shared-memory usage still affects occupancy.

Synchronization

After loading shared memory, all threads must synchronize:

```
__syncthreads();
```

This ensures every thread has written its input value before any other thread reads it.

Inside the scan loop, synchronization is needed between stride steps because stride $2s$ depends on the output of stride s .

A synchronization also acts as a memory-ordering point for shared memory.

Race Condition in In-Place Shared Memory

A tempting update is:

```
if (threadIdx.x >= stride) {  
    buffer_s[threadIdx.x] += buffer_s[threadIdx.x - stride];  
}  
__syncthreads();
```

This is incorrect because it can create a race.

For stride 1, thread 1 writes

$$x_0 + x_1$$

back to location 1. At the same time, thread 2 may need to read the original value

$$x_1$$

from location 1. If thread 1 writes before thread 2 reads, thread 2 reads

$$x_0 + x_1$$

instead of

$$x_1.$$

Then thread 2 computes an incorrect result.

The issue is a **write-after-read** hazard caused by reusing the same buffer for both input and output within the same stride step.

Why Warp-Synchronous Assumptions Are Unsafe

Even if threads are in the same warp, CUDA code should not assume that all threads execute every instruction in perfect lockstep for correctness. Reasons include:

- threads may belong to different warps,
- the compiler may reorder memory instructions,
- values may be promoted to registers,
- Volta and newer architectures support independent thread scheduling,
- the CUDA programming model does not guarantee implicit synchronization among threads for shared-memory communication.

The correct approach is to use synchronization when inter-thread communication through shared memory requires ordering.

Code Walkthrough

Naive In-Place Kogge–Stone Kernel Skeleton

The following version shows the main idea but still contains the race problem if written with direct in-place updates.

```
#define BLOCK_DIM 1024

__global__
void scan_kernel_naive(const float* input,
                      float* output,
                      float* partial_sums,
                      unsigned int n) {
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```

__shared__ float buffer_s[BLOCK_DIM];

if (i < n) {
    buffer_s[threadIdx.x] = input[i];
} else {
    buffer_s[threadIdx.x] = 0.0f;
}

__syncthreads();

for (unsigned int stride = 1;
     stride <= BLOCK_DIM / 2;
     stride *= 2) {

    if (threadIdx.x >= stride) {
        buffer_s[threadIdx.x] =
            buffer_s[threadIdx.x] +
            buffer_s[threadIdx.x - stride];
    }

    __syncthreads();
}

if (i < n) {
    output[i] = buffer_s[threadIdx.x];
}

if (threadIdx.x == BLOCK_DIM - 1) {
    partial_sums[blockIdx.x] = buffer_s[threadIdx.x];
}
}

```

The important indexing logic is:

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

The boundary condition

$$i < n$$

prevents invalid global-memory reads and writes when n is not an exact multiple of the block size.

The condition

$$\text{threadIdx.x} \geq \text{stride}$$

prevents invalid shared-memory reads from negative logical indices.

Race-Free In-Place Version Using a Temporary

To avoid the write-after-read race while still using one shared-memory buffer, split the read and write phases.

```
#define BLOCK_DIM 1024

__global__
void scan_kernel_single_buffer(const float* input,
                              float* output,
                              float* partial_sums,
                              unsigned int n) {
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned int t = threadIdx.x;

    __shared__ float buffer_s[BLOCK_DIM];

    if (i < n) {
        buffer_s[t] = input[i];
    } else {
        buffer_s[t] = 0.0f;
    }

    __syncthreads();

    for (unsigned int stride = 1;
         stride <= BLOCK_DIM / 2;
         stride *= 2) {

        float temp = buffer_s[t];

        if (t >= stride) {
            temp = buffer_s[t] + buffer_s[t - stride];
        }

        __syncthreads();

        if (t >= stride) {
            buffer_s[t] = temp;
        }

        __syncthreads();
    }
}
```

```

    if (i < n) {
        output[i] = buffer_s[t];
    }

    if (t == BLOCK_DIM - 1) {
        partial_sums[blockIdx.x] = buffer_s[t];
    }
}

```

The first synchronization inside the loop ensures all reads from the old buffer state have completed before any thread writes the new state.

The second synchronization ensures all writes from the current stride have completed before the next stride begins.

This version is correct, but it has two synchronizations per stride step.

True Dependence Versus False Dependence

The second synchronization enforces a **true dependence** between iterations:

$$\text{write at stride } s \rightarrow \text{read at stride } 2s.$$

The first synchronization enforces a **false dependence**. The computation does not logically require waiting for all reads before writing, except that the same memory locations are reused for both input and output.

This is a memory-storage conflict, not a mathematical dependence.

Double-Buffered Kogge–Stone Kernel

Double buffering removes the false dependence by using one buffer for reads and another buffer for writes.

```

#define BLOCK_DIM 1024

__global__
void scan_kernel_double_buffered(const float* input,
                                float* output,
                                float* partial_sums,
                                unsigned int n) {
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned int t = threadIdx.x;

    __shared__ float buffer1_s[BLOCK_DIM];
    __shared__ float buffer2_s[BLOCK_DIM];

    float* in_buffer_s = buffer1_s;

```

```

float* out_buffer_s = buffer2_s;

if (i < n) {
    in_buffer_s[t] = input[i];
} else {
    in_buffer_s[t] = 0.0f;
}

__syncthreads();

for (unsigned int stride = 1;
     stride <= BLOCK_DIM / 2;
     stride *= 2) {

    if (t >= stride) {
        out_buffer_s[t] =
            in_buffer_s[t] + in_buffer_s[t - stride];
    } else {
        out_buffer_s[t] = in_buffer_s[t];
    }

    __syncthreads();

    float* temp = in_buffer_s;
    in_buffer_s = out_buffer_s;
    out_buffer_s = temp;
}

if (i < n) {
    output[i] = in_buffer_s[t];
}

if (t == BLOCK_DIM - 1) {
    partial_sums[blockIdx.x] = in_buffer_s[t];
}
}

```

The key idea is:

read from B_{in} , write to B_{out} .

At the end of each stride:

$B_{\text{in}} \leftrightarrow B_{\text{out}}$.

The `else` branch is necessary. Threads with

$$t < \text{stride}$$

do not add anything, but they must still copy their values into the output buffer so the next iteration sees the correct state.

Why Double Buffering Improves Performance

The single-buffer version uses two barriers per loop iteration. The double-buffered version uses one barrier per loop iteration.

For $n_b = \text{BLOCK_DIM}$ elements in a block, the number of stride steps is

$$\log_2 n_b.$$

For $n_b = 1024$,

$$\log_2 1024 = 10.$$

So the single-buffer version uses approximately

$$2 \cdot 10 = 20$$

loop barriers, while the double-buffered version uses

$$10$$

loop barriers.

The lecture reported that for an array of about 2^{25} elements, the double-buffered version reduced runtime from roughly 0.89 milliseconds to about 0.70–0.72 milliseconds on the tested GPU. The important lesson is not the exact number, but the mechanism: eliminating unnecessary synchronization improves latency-bound kernels.

Mathematical Reasoning

Sequential Work

A sequential scan performs one addition per output element after the first. So the exact number of additions for inclusive scan is

$$n - 1.$$

Asymptotically,

$$W_{\text{seq}}(n) = O(n).$$

Kogge–Stone Step Count

Kogge–Stone doubles the stride each step:

$$1, 2, 4, \dots, \frac{n}{2}.$$

The number of steps is

$$S_{\text{KS}}(n) = \log_2 n.$$

Kogge–Stone Work

At stride 2^k , the number of active threads is

$$n - 2^k.$$

Therefore the total number of additions is

$$W_{\text{KS}}(n) = \sum_{k=0}^{\log_2 n - 1} (n - 2^k) \tag{3}$$

$$= n \log_2 n - \sum_{k=0}^{\log_2 n - 1} 2^k \tag{4}$$

$$= n \log_2 n - (n - 1) \tag{5}$$

$$= n \log_2 n - n + 1. \tag{6}$$

Thus

$$W_{\text{KS}}(n) = O(n \log n).$$

This is more work than the sequential algorithm.

Work Efficiency

A parallel algorithm is **work efficient** if it performs the same asymptotic amount of work as the best sequential algorithm.

Since sequential scan does

$$O(n)$$

work but Kogge–Stone scan does

$$O(n \log n)$$

work, Kogge–Stone is not work efficient.

Why Work Inefficiency Can Still Be Acceptable

If the GPU has enough parallel resources, many of the extra additions happen concurrently. The parallel time may still be lower even though total work is higher.

However, if resources are limited, thread blocks serialize. Then the extra work becomes visible as real runtime. This is why work efficiency matters in high-performance GPU code.

Brent–Kung Style Scan

The lecture introduces Brent–Kung as a more work-efficient scan algorithm.

It has two conceptual phases:

1. **reduction phase:** build partial sums up a tree,
2. **post-reduction phase:** propagate partial sums back down to fill missing prefixes.

The reduction phase performs

$$\frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \cdots + 1 = n - 1$$

operations.

The post-reduction phase performs fewer additional operations than Kogge–Stone. The lecture gives the total operation count as

$$2n - \log_2 n - 2.$$

Therefore

$$W_{\text{BK}}(n) = O(n).$$

The step count is approximately

$$S_{\text{BK}}(n) = 2 \log_2 n - 1.$$

So Brent–Kung uses more steps than Kogge–Stone but fewer total additions.

Kogge–Stone Versus Brent–Kung

The trade-off is:

Algorithm	Steps	Work
Kogge–Stone	$\log_2 n$	$O(n \log n)$
Brent–Kung	$2 \log_2 n - 1$	$O(n)$

Kogge–Stone has lower span but more work. Brent–Kung has higher span but better work efficiency.

Performance Intuition

Memory-Bound and Synchronization-Bound Behavior

The scan kernel does little arithmetic per element per stride. Each active thread performs roughly one addition and several shared-memory accesses. Therefore the arithmetic units are not necessarily the bottleneck.

Important bottlenecks include:

- shared-memory latency,
- synchronization latency,
- extra work from non-work-efficient algorithms,
- global-memory traffic for input, output, and partial sums,
- kernel launch overhead for multi-block scan.

Why Shared Memory Helps

Without shared memory, every stride step would read and write global memory. For Kogge–Stone, this would cause approximately

$$O(n \log n)$$

global-memory traffic, which would be extremely expensive.

Using shared memory changes the global-memory traffic to roughly:

one global load per element + one global store per element + partial sum traffic.

The iterative work happens in shared memory, which is much lower latency and higher bandwidth than global memory.

Why Synchronization Is Expensive

Each `__syncthreads()` forces all threads in a block to wait. In a latency-bound kernel, barriers can dominate runtime because there is not enough independent arithmetic to hide their cost.

The single-buffer version uses two barriers per stride because it must separate read and write phases. Double buffering removes the false dependence and reduces the barrier count.

Occupancy Considerations

Occupancy depends on:

- threads per block,
- registers per thread,
- shared memory per block,
- hardware limits on resident blocks per SM.

Using double buffering doubles shared-memory usage. This may reduce occupancy if shared memory becomes the limiting resource. Therefore double buffering is not automatically free. It trades shared memory for fewer synchronizations.

For a block of 1024 threads and two float buffers:

$$\text{shared memory per block} = 2 \cdot 1024 \cdot 4 = 8192 \text{ bytes.}$$

This is modest, but for larger types, larger tiles, or multiple arrays, shared-memory pressure can matter.

Boundary Handling for Non-Power-of-Two Sizes

The lecture notes that if the global input size is not an exact multiple of the block dimension, only the last block needs special handling.

Instead of physically padding the input array, the kernel can pad on the fly:

$$\text{if } i \geq n, \quad \text{load } 0.$$

For addition scan, zero is the identity value. More generally, for operator $\oplus$, the padding value should be the identity element e .

When writing back:

$$\text{if } i < n, \quad \text{store output.}$$

This avoids invalid global-memory accesses.

Bank Conflicts

Although the lecture does not go deeply into shared-memory bank conflicts, scan algorithms can produce bank conflicts because neighboring threads read from addresses separated by powers of two. On older GPUs, this was a major performance issue. On modern GPUs, the hardware is more forgiving, but bank conflicts can still matter in carefully optimized scan kernels.

Common Misconceptions

Misconception: Scan Is Just Reduction

Reduction produces one value:

$$r = x_0 + x_1 + \cdots + x_{n-1}.$$

Scan produces every prefix:

$$[y_0, y_1, \dots, y_{n-1}].$$

Scan can use reduction-like trees internally, but it is a richer primitive.

Misconception: One Synchronization at the End Is Enough

One barrier after the loop is not sufficient because each stride depends on values produced by the previous stride.

The dependence is:

$$\text{stride } s \rightarrow \text{stride } 2s.$$

Therefore synchronization is needed between stride steps.

Misconception: Threads in a Warp Are Always Safe Without Synchronization

Warp-level execution details can help performance reasoning, but correctness should follow the CUDA programming model. Shared-memory communication between threads requires explicit synchronization unless using carefully designed warp-level primitives with the appropriate masks and semantics.

Misconception: More Parallelism Always Means Faster

Kogge–Stone exposes more parallelism and has fewer steps, but it performs

$$O(n \log n)$$

work. Brent–Kung performs

$$O(n)$$

work but has more steps. Depending on hardware, problem size, and bottlenecks, either style may be preferable.

Misconception: Double Buffering Is Only for Scan

Double buffering is a general optimization. It appears in tiled matrix multiplication, convolution, data staging pipelines, and asynchronous copy patterns. The general idea is:

compute using buffer A while preparing or writing buffer B.

In this lecture, double buffering removes a false dependence rather than overlapping global-memory copy with compute, but the underlying principle is similar.

Practical Takeaways

- Scan is a foundational GPU primitive for offset computation, stream compaction, allocation, radix sort, histograms, sparse data structures, and parallel partitioning.
- Sequential scan is simple but has a loop-carried dependence.
- A full-array GPU scan is usually decomposed into per-block scans, scan of block partial sums, and a correction pass.
- Kogge–Stone scan uses stride doubling and completes in $\log_2 n$ steps.
- Kogge–Stone is not work efficient because it performs $O(n \log n)$ additions.
- Shared memory is essential for avoiding repeated global-memory traffic.
- In-place shared-memory updates can introduce races.
- Correctness requires separating reads and writes or using double buffering.
- Double buffering removes a false dependence and can reduce synchronization overhead.
- Brent–Kung scan performs $O(n)$ work but uses more synchronization steps.

- The best scan algorithm depends on the balance between parallelism, synchronization cost, memory traffic, and work efficiency.

Project or Experiment Ideas

1. Implement sequential CPU inclusive scan and compare it against CUDA per-block scan.
2. Implement the single-buffer Kogge–Stone kernel with two synchronizations per stride.
3. Implement the double-buffered Kogge–Stone kernel and measure the effect of removing one barrier.
4. Benchmark block sizes 128, 256, 512, and 1024.
5. Test non-power-of-two input sizes using identity-value padding.
6. Implement exclusive scan by shifting the inclusive result or by modifying the load and store logic.
7. Implement Brent–Kung scan and compare against Kogge–Stone.
8. Use Nsight Compute to measure barrier stalls, shared-memory throughput, global-memory throughput, occupancy, and achieved bandwidth.
9. Compare a custom scan implementation against CUB’s scan primitive.
10. Extend scan to practical applications such as stream compaction, where each element emits either zero or one output item.

Final Mental Model

Scan is the GPU primitive that turns local information into global positions. The sequential version is a simple recurrence, but the recurrence creates a loop-carried dependence. Parallel scan breaks this dependence by restructuring the computation into tree-like stages.

Kogge–Stone scan is like every element repeatedly reaching backward by powers of two:

$$1, 2, 4, 8, \dots$$

After each step, every element knows a wider prefix region. This gives low step count but extra work. Shared memory makes the repeated communication fast, but synchronization and races must be handled carefully. Double buffering removes the false dependence caused by reusing the same buffer for input and output.

The deeper engineering lesson is:

Parallel GPU algorithms are not only about exposing threads. They are about managing dependences, memory hierarchy, synchronization, and total work so that the hardware performs useful work instead of waiting, racing, or doing unnecessary operations.